

Large Reasoning Models

Post-Training with GRPO and GSPO

Naeemullah Khan

naeemullah.khan@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية

King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

May 27, 2026

1. Motivation and learning outcomes
2. Large reasoning models: definitions, LLMs vs. LRMs, and types of reasoning
3. Prompting for reasoning: Chain-of-Thought, Tree-of-Thought, and Self-Consistency
4. Instruction tuning for reasoning tasks (FLAN and related methods)
5. RL foundations for post-training: policies, PPO, and DPO
6. GRPO and GSPO for reasoning-model alignment
7. DeepSeek-R1: training pipeline, rewards, emergent behavior, and key findings
8. Summary and references

► Why study LRMs?

- LLMs like GPT-3 and GPT-4 excel at generating text.
- However, they often rely on **pattern matching**, not true reasoning.
- Real-world applications like **math problem-solving**, **code synthesis**, and **scientific hypothesis generation** require multi-step reasoning.

► Examples:

- *GPT-3 can generate realistic text but fails at solving "If Alice has 3 apples and gives Bob 2, how many does she have?" unless prompted with reasoning steps.*
- *DeepMind's AlphaCode uses reasoning and symbolic techniques to solve programming problems—pure LLMs fall short.*

By the end of this session, you should be able to:

- ▶ Define and distinguish Large Reasoning Models (LRMs) from simpler pattern-matching LLMs.
- ▶ Understand advanced reasoning techniques, including Chain-of-Thought (CoT), Tree-of-Thought (ToT), and self-consistency methods.
- ▶ Explain how instruction tuning and reinforcement learning (RL) can refine and enhance reasoning abilities in language models.
- ▶ Derive the intuition behind **GRPO**, **GSPO** and the differences between them.
- ▶ Critically assess future directions and identify open problems in the field of reasoning with language models.

What Are Large Reasoning Models (LRMs)?

► Definition:

- Large Language Models (LLMs) optimized for **multi-step logical reasoning**
- Emphasize Chain-of-Thought (CoT) style reasoning

► Key Examples:

- OpenAI: o1, o3
- Google: Gemini 2.0 (Flash mode)
- DeepSeek: R1, V3
- QwQ (Alibaba)
- Claude 3.5
- Codestral (Mistral)

► Distinctive Behavior:

- “Thinking internally”
- Producing hidden or explicit reasoning traces
- Step-by-step intermediate computations
- Intermediate reasoning traces (explicit or latent)
- Better performance on symbolic tasks and math
- Often paired with advanced prompting or finetuning

► Examples:

- **PaLM + CoT Prompting:**

Google's PaLM + CoT prompt significantly improved accuracy on GSM8K (grade-school math) from approx 18% to 58%.

- **Gemini 2.0 Flash mode:**

Provides transparent reasoning steps for decision-making.

- **DeepSeek R1:**

Combines expert models for efficient coding and debugging.

Claimed to achieve "Aha!" moments, solving reasoning tasks using structured thinking steps, not just memorization.

Large language models (LLMs) and large reasoning models (LRMs) share foundational technologies but serve different purpose:

► Core Function:

- **LLMs:** Learn patterns in data to generate fluent, human-like text.
- **LRMs:** Extend LLMs to solve problems requiring logical reasoning and contextual understanding.

► Use Cases:

- **LLMs:** Content generation, language translation, summarization, and answering user queries.
- **LRMs:** Math problem solving, interpreting ambiguous clinical data, and complex decision-making.

► Reasoning Ability:

- **LLMs:** Limited structured reasoning; may struggle with multi-step logic or ambiguity.
- **LRMs:** Trained to apply consistent reasoning steps; outputs are more logical and verifiable.

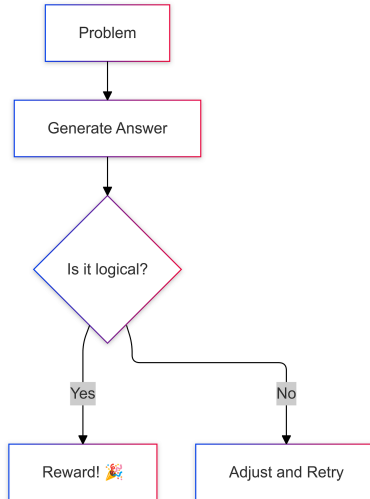
► Performance:

- **LLMs:** Fast response times; optimized for scalability and speed.
- **LRMs:** Slower response times; require more processing to reason through problems step by step.

► Best Suited For:

- **LLMs:** High-volume, low-risk tasks where speed is critical.
- **LRMs:** High-stakes or complex tasks where accuracy, explainability, and logic matter.

How Do LRMs Work?



LRMs use a combination of training methods and prompt strategies to enhance the reasoning capabilities of LLMs.

► Training on Enriched Datasets

- Includes reasoning-focused examples
- Real-world scenarios with clear outcomes
- Teaches both correct answers and reasoning steps

► Reinforcement Learning (RL)

- Rewards for logical, correct answers
- Penalties for incorrect or inconsistent reasoning
- Reinforces desirable reasoning patterns

► Human Feedback (RLHF)

- Human reviewers guide model outputs

- Refines nuanced reasoning strategies
- Useful in domains needing expertise (e.g., healthcare, finance)

► **Prompt Engineering**

- Carefully designed prompts activate reasoning
- Guides model through multi-step or layered tasks
- Generic prompts may not trigger deep reasoning

► **Chain-of-Thought (CoT) Prompting**

- Breaks problems into smaller steps
- Explores multiple reasoning paths
- Improves transparency and accuracy
- Essential for auditability and explainability

Large reasoning models (LRMs) employ several types of reasoning to simulate or support human-like problem solving:

► Deductive Reasoning

- Applies general rules to specific cases for logically certain conclusions (top-down reasoning).
- Excels at structured, rule-based tasks requiring high logical consistency.
- Example domains: regulatory compliance, medical protocol analysis.
- *Example:*
Rule: All patients with a fever and cough should be tested for influenza.
Case: Patient A has a fever and cough.
Deduction: Patient A should be tested for influenza.

► Inductive Reasoning

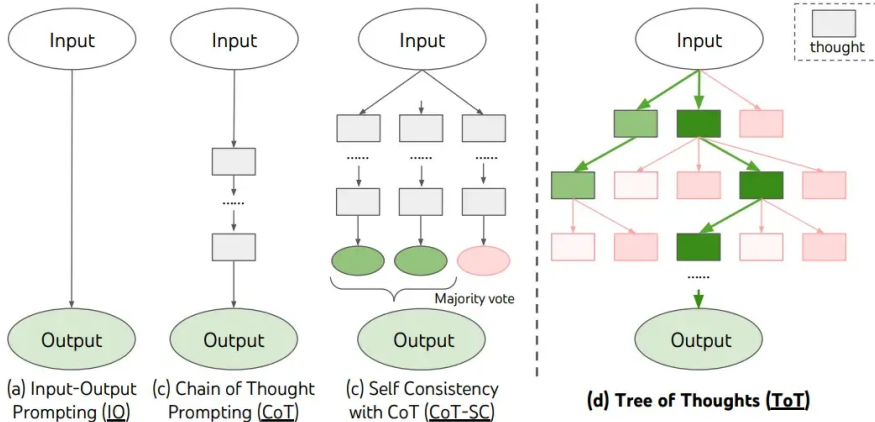
- Draws general conclusions from specific observations in data.
- Supports probabilistic predictions and generalization to unseen inputs.
- Useful in dynamic, data-rich scenarios (e.g., fraud detection).
- *Example:*
Observation: 90% of emails with suspicious links are phishing attempts.
Induction: An email with a suspicious link is likely to be a phishing attempt.

► Abductive Reasoning

- Infers the most likely explanation from incomplete or uncertain evidence.
- Generates hypotheses rather than definitive answers.
- Effective in domains like medical diagnostics, where plausible interpretations are needed.
- *Example:*
Evidence: The lawn is wet in the morning.
Possible explanations: It rained overnight, or the sprinkler was on.
Abduction: If the weather report says no rain, the most likely explanation is the sprinkler was on.

► Analogical Reasoning

- Identifies similarities between different situations or datasets.
- Transfers insights from one context to another for novel problem solving.
- Example: recommending products based on similar customer behavior.
- *Example:*
Situation A: A student learns to solve quadratic equations by factoring.
Situation B: The student applies the same factoring method to solve cubic equations.
Analogy: The approach used for quadratics is adapted to cubics.



Comparison of prompting strategies: IO, Chain-of-Thought, Self-Consistency, and Tree of Thoughts. Source: Yao et al. (2023), [arXiv:2305.10601](https://arxiv.org/abs/2305.10601).

Reasoning Techniques in LRMs (cont.)

► What is CoT?

- A prompting technique that encourages LLMs to generate intermediate reasoning steps before arriving at a final answer.
- Helps models to break down complex problems into manageable parts.

► How it works:

- Instead of asking for a direct answer, the prompt includes examples that show the reasoning process.
- The model is guided to think step-by-step, mimicking human-like reasoning.

► Benefits:

- Improves accuracy on tasks requiring multi-step reasoning, such as math problems or logical puzzles.
- Makes the model's reasoning process more transparent, allowing users to follow its logic.

► Example:

- Instead of asking "What is 24 divided by 6?", the prompt might be "To find 24 divided by 6, we can think: 6 goes into 24 how many times? Let's break it down step by step."

► What is Few-shot CoT?

- A variant of CoT prompting that provides a few examples of reasoning steps before the actual question.
- Helps the model learn the reasoning pattern from the examples.

► How it works:

- The prompt includes several examples of problems and their step-by-step solutions.
- The model uses these examples to understand how to approach the new problem.

► Benefits:

- Enhances the model's ability to generalize from examples, improving performance on similar tasks.
- Reduces the need for extensive training on specific reasoning tasks.

► Example:

- The prompt might include examples like "If 6 times 4 equals 24, then what is 6 times 5?" followed by the reasoning steps leading to the answer.
- The model learns to apply similar reasoning to new problems presented in the same format.

► What is Zero-shot CoT?

- A variant of CoT prompting where the model is prompted to reason through a problem without any prior examples.
- The model generates its own reasoning steps based on the problem statement.

► How it works:

- The prompt directly asks the model to think through the problem step-by-step, without providing specific examples.
- The model relies on its training to generate a logical sequence of reasoning.

► Benefits:

- Useful for novel problems where no examples are available.
- Encourages the model to apply its learned reasoning skills flexibly.

► Example:

- The prompt might be "To solve the problem of how many apples Alice has after giving some to Bob, think step-by-step about what happens when she gives away apples."
- The model generates its own reasoning steps based on the prompt.

► What is Auto-CoT?

- An automated approach to generating Chain-of-Thought prompts.
- Uses algorithms to create reasoning steps based on the problem context.

► How it works:

- The system analyzes the problem and automatically generates a sequence of reasoning steps.
- This can be done using techniques like reinforcement learning or heuristic methods.

► Benefits:

- Reduces the manual effort required to create effective CoT prompts.
- Can adapt to a wide range of problems without needing specific examples.

► Example:

- The system might take a problem like "If Alice has 3 apples and gives 2 to Bob, how many does she have left?" and automatically generate reasoning steps like "Start with 3 apples, subtract 2 apples given to Bob, and conclude with the remaining number of apples."

► What is ToT?

- A reasoning technique that structures the thought process as a tree, allowing for exploration of multiple reasoning paths.
- Each node in the tree represents a step in the reasoning process, and branches represent different possible paths or decisions.

► How it works:

- The model generates a tree structure where each branch represents a different reasoning path.
- The model can explore multiple paths simultaneously, evaluating the outcomes of each.

► Benefits:

- Allows for more complex reasoning by considering multiple possibilities at once.
- Helps the model avoid getting stuck in a single line of reasoning, improving problem-solving capabilities.

► Example:

- For a problem like "If Alice has 3 apples and gives some to Bob, how many does she have left?", the model might create a tree with branches for different amounts given to Bob (e.g., 1, 2, or 3 apples) and explore the outcomes of each branch.

► What is Self-Consistency?

- A technique that improves the reliability of LLM outputs by generating multiple responses to the same prompt and selecting the most consistent answer.
- Helps mitigate the variability in LLM responses, especially in tasks requiring reasoning or complex decision-making.

► How it works:

- The model generates several responses to a given prompt.
- These responses are then evaluated for consistency, and the most frequent or coherent response is selected as the final answer.

► Benefits:

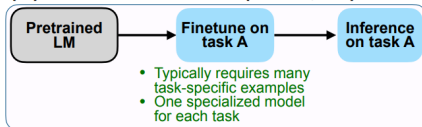
- Reduces the impact of noise and randomness in LLM outputs, leading to more reliable results.
- Enhances the model's performance on tasks that require stable and consistent reasoning.

► Example:

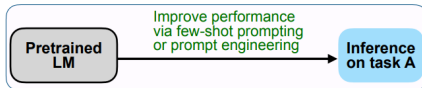
- For a question like "What is the capital of France?", the model might generate multiple responses such as "Paris", "Paris is the capital", and "The capital city is Paris". The most consistent response ("Paris") would be selected as the final answer.

Instruction Tuning for Reasoning Tasks

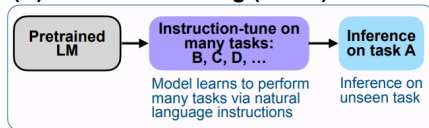
(A) Pretrain–finetune (BERT, T5)



(B) Prompting (GPT-3)



(C) Instruction tuning (FLAN)



Three ways to use a language model: pretrain–finetune, prompting, and instruction tuning (FLAN). Source: Wei et al. (2022), [arXiv:2109.01652](https://arxiv.org/abs/2109.01652).

► What is Instruction Tuning?

- Fine-tuning LLMs on a dataset of instruction-response pairs.
- Enhances the model's ability to follow instructions and perform tasks.

► Why is it important?

- Improves model performance on diverse tasks.
- Enables better generalization to unseen tasks.

Why Instruction Tune LLMs?

- ▶ Pre-trained LLMs are optimized for next-word prediction, not for following instructions or engaging in conversations.
- ▶ Without instruction tuning, LLMs may generate text that is linguistically correct but not aligned with user intent.
- ▶ Instruction tuning uses supervised learning on instruction-response pairs, teaching the model to generate helpful and relevant outputs for user prompts.
- ▶ Fine-tuning is much more efficient than training a large model from scratch and can be performed with less data and compute.
- ▶ Instruction tuning bridges the gap between generic language modeling and practical, task-oriented use cases, making LLMs more useful and predictable.

Summary of Instruction Tuning

- ▶ **Definition:** Fine-tuning LLMs on diverse instruction-response pairs.
- ▶ **Key Elements:**
 - *Instruction:* Task description in natural language.
 - *Additional Information:* Optional context or input.
 - *Desired Output:* Target response for evaluation.
- ▶ **Benefits:**
 - Improves ability to follow instructions.
 - Reduces need for prompt engineering and few-shot examples.
 - Enhances generalization to unseen tasks.
- ▶ **Applications:**
 - Translation, question answering, reading comprehension, NLI.
 - Broad improvements across reasoning and practical tasks.

Why Does Instruction Tuning Work?

- ▶ Models learn to **follow instructions step by step**, rather than just predicting the next word or memorizing answers.
- ▶ Encourages **logical reasoning**: Models are guided to explain their answers or show their work, which helps in tasks like math, logic puzzles, and multi-step questions.
- ▶ **Generalization**: Instruction-tuned models can handle new types of questions or tasks they have not seen before, simply by following the provided instruction.
- ▶ **Variety of Examples**:
 - *Math*: "Solve: What is 15 divided by 3?" → "5"
 - *Reasoning*: "Is a whale a mammal or a fish? Explain your answer." → "A whale is a mammal because it gives birth to live young and breathes air."
 - *Summarization*: "Summarize the following paragraph in one sentence."

Challenges and Limitations of Instruction Tuning

► Dataset Quality and Diversity:

- Large, high-quality datasets are costly to create.
- Reliance on few open-source datasets may reduce diversity.

► Bias Propagation:

- Proprietary LLMs used for instruction generation can spread their biases to open-source models.

► Evaluation Concerns:

- Evaluation often uses proprietary models, which may not reveal all biases or limitations.

► Superficial Learning:

- Smaller models may mimic style without true reasoning.
- Gains may reflect pattern matching, not understanding.

► Overfitting to Training Tasks:

- Evaluation tasks too similar to training can overstate improvements.

► Key Insight:

- Improving base model quality is more impactful than just imitating proprietary systems.

Reinforcement Learning for Reasoning Skill Refinement



► What is Reinforcement Learning for Reasoning Skill Refinement?

- A method to enhance reasoning skills in LLMs through reinforcement learning.
- Focuses on refining the model's ability to perform reasoning tasks.

► How does it work?

- Uses a reward signal to guide the model towards better reasoning performance.
- Involves training the model on a variety of reasoning tasks with feedback.



► Why is it important?

- Enhances the model's reasoning capabilities, making it more effective for complex tasks.
- Addresses the limitations of traditional training methods by focusing on skill refinement.

► Challenges

- Requires a well-defined reward signal to effectively guide the learning process.
- Balancing exploration and exploitation during training can be difficult.

- ▶ A **policy** π specifies how an agent (or model) chooses actions.
- ▶ **Deterministic:** $\pi(s) = a$.
- ▶ **Stochastic:** $\pi(a | s) = \Pr(a_t = a | s_t = s)$.
- ▶ For LLMs: s is the prompt/context, a is the next token; the policy is $\pi_\theta(\text{token} | \text{context})$.
- ▶ **Goal:** find π^* that maximizes expected cumulative reward (or, in RLHF, human-aligned reward).

- ▶ Value-based methods (Q-learning) can be hard in huge state spaces.
- ▶ Often easier to parameterize a policy class $\Pi = \{\pi_\theta \mid \theta \in \mathbb{R}^m\}$.
- ▶ Maximize expected return $\mathcal{J}(\theta) = \mathbb{E}[\sum_{t \geq 0} \gamma^t r_t \mid \pi_\theta]$.
- ▶ **Policy gradient:** update θ along $\nabla_\theta \mathcal{J}(\theta)$ (gradient ascent on parameters).
- ▶ PPO, DPO, and GRPO are all policy-gradient-style updates on language-model weights.

- ▶ **Problem:** large policy updates can collapse performance.
- ▶ **Idea:** take the biggest safe improvement step on the current batch of data.
- ▶ **Clipped objective** (widely used variant): with $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_k}(a_t|s_t)}$,

$$\mathcal{L}^{\text{CLIP}} = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right]$$

- ▶ ϵ is small (e.g., 0.2)—limits how far the new policy can drift per update.

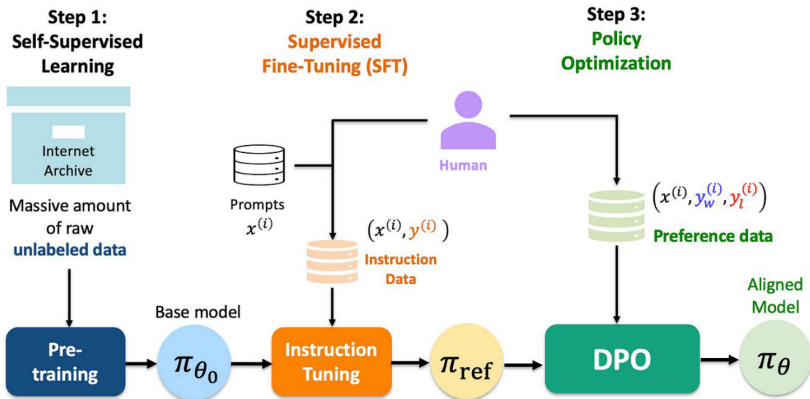
- ▶ **Stable** optimization with clipped updates.
- ▶ Encourages exploration of phrasings at each rollout.
- ▶ **Expensive:** many rollouts, critic + RM + reference in memory.
- ▶ **Sensitive** to reward-model mistakes and distribution shift.

- ▶ **Bypasses** explicit RM training and RL rollouts (Rafailov et al., 2023).
- ▶ Optimize the policy **directly** from preference pairs (y^+, y^-) .
- ▶ **DPO loss** (simplified form):

$$L_{\text{DPO}} = -\log \sigma \left(\beta \log \frac{\pi_{\theta}(y^+)}{\pi_0(y^+)} - \beta \log \frac{\pi_{\theta}(y^-)}{\pi_0(y^-)} \right)$$

- ▶ π_0 = frozen reference (SFT) policy; β = temperature.
- ▶ Often **simpler and more stable** than PPO for alignment—but still preference-based, not verifiable math/code rewards.

Direct Preference Optimization (DPO)



DPO pipeline: pre-training, supervised fine-tuning, then direct preference optimization from human preference pairs. Source: Rafailov et al. (2023), [arXiv:2305.18290](https://arxiv.org/abs/2305.18290)

- ▶ **GRPO**: General Reinforced Preference Optimization, an evolution of PPO that is more stable, efficient, and converges faster
- ▶ Combines reward and preference signals, using single model for both
- ▶ Improves safety and alignment of open-weight models
- ▶ Used in DeepSeek-MoE and other models

- ▶ GRPO avoids the need for a separate critic model by estimating the baseline from group scores.
- ▶ For each question q , sample a group of outputs $\{o_1, o_2, \dots, o_G\}$ from the old policy $\pi_{\theta_{\text{old}}}$.
- ▶ The policy model π_{θ} is optimized by maximizing:

$$J_{\text{GRPO}}(\theta) = \mathbb{E}_{q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(O|q)} \left[\frac{1}{G} \sum_{i=1}^G \left(\min \left(\frac{\pi_{\theta}(o_i|q)}{\pi_{\theta_{\text{old}}}(o_i|q)} A_i, \right. \right. \right. \\ \left. \left. \left. \text{clip} \left(\frac{\pi_{\theta}(o_i|q)}{\pi_{\theta_{\text{old}}}(o_i|q)}, 1 - \epsilon, 1 + \epsilon \right) A_i \right) - \beta D_{\text{KL}}(\pi_{\theta} || \pi_{\text{ref}}) \right) \right] \quad (1)$$

- KL divergence term:

$$D_{\text{KL}}(\pi_{\theta} || \pi_{\text{ref}}) = \frac{\pi_{\text{ref}}(o_i | q)}{\pi_{\theta}(o_i | q)} - \log \frac{\pi_{\text{ref}}(o_i | q)}{\pi_{\theta}(o_i | q)} - 1 \quad (2)$$

- Advantage A_i is computed using group rewards $\{r_1, r_2, \dots, r_G\}$:

$$A_i = \frac{r_i - \text{mean}(\{r_1, r_2, \dots, r_G\})}{\text{std}(\{r_1, r_2, \dots, r_G\})} \quad (3)$$

- ϵ and β are hyper-parameters.

- ▶ **Fewer models in memory:** policy + reference (no critic, no RM when rules suffice).
- ▶ **Often faster to train** than PPO-style RLHF on comparable hardware.
- ▶ Powers DeepSeekMath, DeepSeek-R1-Zero, and many open reproductions.
- ▶ With QLoRA / optimized kernels, 8B-scale GRPO fits consumer GPUs (order-of-magnitude below naive full fine-tune).

Setup (Llama 8B, indicative)	Naive	Optimized	Save
GRPO (full)	~510 GB	~160 GB	~68%
GRPO + QLoRA	—	<10 GB	>90%

	PPO (RLHF)	GRPO (RLVR)
Baseline	Critic / value net	Mean reward in a group
Reward	Learned RM	Rules / verifiers
Models in memory	Policy, ref., RM, critic	Policy, ref. (typical)
Best for	Chat alignment	Math, code, logic

- ▶ GRPO does **not** replace DPO—DPO still wins when you only have preference pairs and no verifier.

- ▶ Long RL runs on huge models need **stable** updates—unstable RL $\Rightarrow$ **model collapse** (irreversible).
- ▶ **GRPO** uses **token-level** importance ratios and **per-token** clipping on long reasoning chains.
- ▶ Clipping can still leave many tokens with skewed weights; effects compound over training.
- ▶ On **MoE** models, token-level updates amplify **routing churn** (experts change mid-training)—often needs extra fixes (e.g. Routing Replay).

- ▶ Rewards are usually **sequence-level** (correct answer, tests passed)—optimization should match that granularity.
- ▶ Same group-relative advantages as GRPO: compare G completions on the same prompt.
- ▶ Design goals ([Qwen blog](#)):
 - **Stable**—especially for MoE RL;
 - **Efficient**—better compute-to-gain than GRPO;
 - **Infrastructure-friendly**—more robust to numeric precision issues in distributed RL.

- ▶ **Sample** a batch of prompts $x \sim \mathcal{D}$.
- ▶ For each x , draw a **group** $\{y_i\}_{i=1}^G$ from the old policy $\pi_{\theta_{\text{old}}}$.
- ▶ **Reward** each full response (verifier / rule checker) $\Rightarrow$ group-relative advantage $\hat{A}_i$ (same formula as GRPO).
- ▶ Build a **sequence-level** importance weight $s_i(\theta)$ from $\pi_{\theta}(y_i|x)$ vs. $\pi_{\theta_{\text{old}}}(y_i|x)$.
- ▶ **Clip and optimize once per sequence** (not token-by-token); optional KL to π_{ref} as in PPO/GRPO stacks.
- ▶ **Variant:** GSPO-token allows per-token advantages when needed (e.g. multi-turn RL).

	GRPO	GSPO
Importance ratio	Per-token	Sequence-level $s_i(\theta)$
Clipping	Token-by-token	Once per sequence
Advantage	Group-relative	Group-relative (same)
Reward alignment	Mismatch on long CoT	Matches sequence rewards
MoE RL	Often unstable	Stable in Qwen3-scale runs
Typical use	DeepSeek-R1, dense RL	Qwen3, large MoE RL

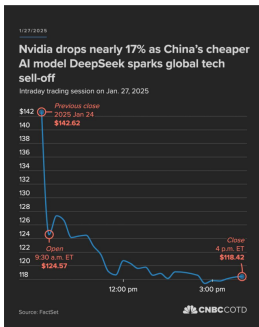
- GSPO can clip **more tokens** in aggregate yet train **more efficiently**—lower-variance signal.



Nvidia CEO Jensen Huang is impressed by the Chinese company that made Nvidia 'poorer' by \$580 billion in one single day

TOI Newscard Team / TIMESOFINDIA.COM / Updated: Jan 06, 2026, 16:37 IST

Select **TOI** as Comments Share AA



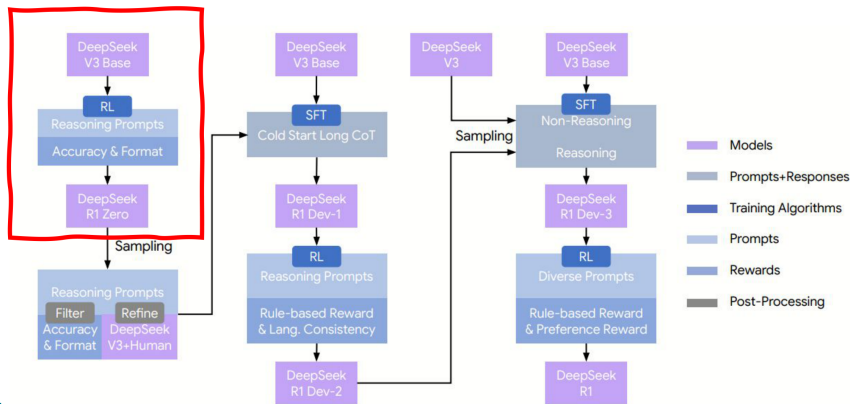
Nvidia's CEO Jensen Huang lauded Chinese AI startup DeepSeek for its role in advancing open-source AI, despite its model causing a massiv..

Nvidia CEO Jensen Huang has nothing but admiration for DeepSeek—the Chinese AI startup that wiped nearly \$600 billion off his company's market value in a single day last January. Speaking at the Consumer Electronics Show in Las Vegas on January 5, Huang credited DeepSeek with "activating" a global shift toward open-source artificial intelligence, calling the company's work "really, really exciting" and saying Nvidia was "so happy with it."

The praise comes almost a year after DeepSeek's R1 model sparked panic among investors in January 2025.

The Chinese startup claimed it had developed a competitive AI model using just 2,048 of Nvidia's older H800 chips—costing under \$6 million and taking only two months. By comparison, US companies were spending tens of millions on thousands of Nvidia's most advanced chips for similar results.

The news triggered a bloodbath on Wall Street. Nvidia's stock crashed 17% on January 27, 2025, and the



DeepSeek-R1 training pipeline overview. Source: Choi (2026) [arXiv:2501.12948](https://arxiv.org/abs/2501.12948).

Table 2 | An interesting “aha moment” of an intermediate version of DeepSeek-R1-Zero. The model learns to rethink using an anthropomorphic tone. This is also an aha moment for us, allowing us to witness the power and beauty of reinforcement learning.

Question: If $a > 1$, then the sum of the real solutions of $\sqrt{a - \sqrt{a + x}} = x$ is equal to

Response: <think>

To solve the equation $\sqrt{a - \sqrt{a + x}} = x$, let's start by squaring both ...

$$\left(\sqrt{a - \sqrt{a + x}}\right)^2 = x^2 \implies a - \sqrt{a + x} = x^2.$$

Rearrange to isolate the inner square root term:

$$(a - x^2)^2 = a + x \implies a^2 - 2ax^2 + (x^2)^2 = a + x \implies x^4 - 2ax^2 - x + (a^2 - a) = 0$$

...

Wait, wait. Wait. That's an aha moment I can flag here.

Let's reevaluate this step-by-step to identify if the correct sum can be ...

We started with the equation:

$$\sqrt{a - \sqrt{a + x}} = x$$

First, let's square both sides:

$$a - \sqrt{a + x} = x^2 \implies \sqrt{a + x} = a - x^2$$

Next, I could square both sides again, treating the equation: ...

...



Accuracy Reward

For math: compare final answer against ground truth. For code: run test cases. The reward is essentially binary — correct or not.



Format Reward

A small reward for placing reasoning inside `<think>...</think>` tags and the final solution afterwards.



No Process Reward Model (PRM)

DeepSeek argues that PRMs are expensive, subject to reward hacking, and may constrain the model's ability to discover novel reasoning strategies. Outcome-based rewards give maximum freedom.

Caveat: additional composite rewards (e.g., language consistency, safety, ...) added to the later stage RL leading to the final R1



Self-verification

The model checks its own work after deriving an answer. Emerges because self-checked outputs are more often correct.



Reflection and backtracking

“Wait, let me reconsider” or “That approach doesn’t work because...”
The model recognizes wrong paths and restarts with a different strategy.



Extended Deliberation

The response length grows from hundreds to thousands of tokens. The model is essentially learning that thinking longer leads to better answers—a learned form of test-time compute scaling.



Poor readability

Reasoning traces are often poorly formatted and are difficult for humans to follow.



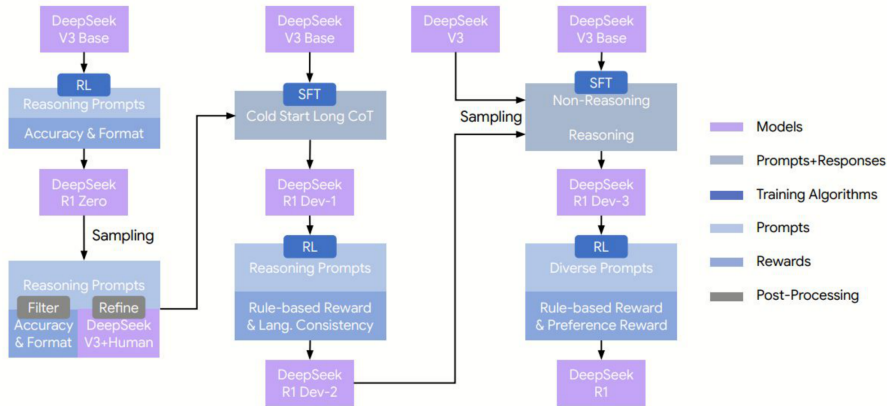
Code switching

The model frequently switches between languages within a single response, regardless of the input language.



Narrow scope, no general capability or safety discovery outside RLVR

Pure RL only works well for tasks with verifiable answers (math, code). Open-ended tasks like creative writing or general conversation are not effectively addressed. Also, R1-Zero won't discover safety guardrails by solving math and code...



► Finding 1: RL alone can induce reasoning

- **Prior assumption:** chain-of-thought reasoning required supervised examples.
- **Finding:** R1-Zero demonstrates that outcome-based RL, with no process supervision, can induce sophisticated reasoning capabilities.
- **Caveat 1:** the final R1 is still going through the SFT→RL pipeline in the end
- **Caveat 2:** doesn't apply to small (less capable) models

► Finding 2: Outcome rewards can enable discovery

- **Prior assumption:** sophisticated process rewards are necessary for O1 reasoning.
- **Finding:** Process reward models can inadvertently constrain exploration by penalizing unconventional yet effective reasoning paths. The lesson: sometimes less supervision yields more capability.
- **Caveat:** RLVR is not applicable for all reasoning problems.

► Finding 3: RL + SFT synergy

- Neither pure RL (R1-Zero) nor pure SFT produces the best results.
- The multi-stage pipeline shows that RL and SFT play complementary roles: RL discovers capability, SFT provides reliability.
- Cold-start data prevents early RL instability; rejection sampling converts RL discoveries into reusable training data.

► Finding 4: Distillation outperforms direct RL on small models

- If you have a limited compute budget, you are better off distilling from a large reasoning model than applying RL to a small model directly.
- Through distillation, reasoning capabilities transfer to models as small as 1.5B parameters, making strong reasoning accessible at any scale.

► Finding 4: Distillation outperforms direct RL on small models

- If you have a limited compute budget, you are better off distilling from a large reasoning model than applying RL to a small model directly.
- Through distillation, reasoning capabilities transfer to models as small as 1.5B parameters, making strong reasoning accessible at any scale.

► Finding 5: Open-weight reasoning models are viable

- R1 models are released under an MIT license.
- The distilled 7B and 14B models outperform many much larger closed-source models on reasoning benchmarks.

► Finding 6: Structured search algorithms for inference scaling?

- **Prior assumption:** Inference-time search algorithm such as Monte-Carlo Tree Search might be necessary.
- **Findings:** Reasoning can be entirely autoregressive; no special search algorithm necessary.
- **Caveat:** Later models for hard math benchmarks do incorporate structure, though not in the traditional MCTS sense.

► Finding 7: Test-time compute is a learnable resource

- **Findings:** R1's models learn to allocate more thinking tokens to harder problems. This means test-time compute scaling is not just an inference trick (like beam search or majority voting)—it can be internalized by the model itself through training. The model learns when and how much to think.
- **Caveat:** true to some degree; later studies reveal challenges to address.

- ▶ **Reasoning models** spend test-time compute on chains of thought; post-training (especially RLVR) is what makes this reliable.
- ▶ **GRPO** removes the critic, uses group-relative advantages, and pairs naturally with verifiable rewards—the backbone of DeepSeek-R1-Zero.
- ▶ **DeepSeek-R1** combines cold-start SFT, reasoning RL, synthetic SFT, and alignment RL; **distillation** transfers gains to smaller models.
- ▶ **GSPO** stabilizes long-sequence and MoE RL via sequence-level clipping; newer methods (SAPO, HölderPO) refine credit assignment.
- ▶ Watch for **plateaus**, **looping**, and **calibration**—scaling laws and failure modes differ from pretraining.

GSPO

- ▶ Zheng, C., Liu, S., Li, M., et al. (Qwen Team, Alibaba) (2025). *Group Sequence Policy Optimization*. [arXiv:2507.18071](#) / [PDF](#).
- ▶ Qwen Team (2025). *GSPO: Towards Scalable Reinforcement Learning for Language Models* (blog). [qwen.ai/blog?id=gspe](#).

GRPO, reasoning models, and RL

- ▶ DeepSeek-AI (2025). *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*. [arXiv:2501.12948](#).
- ▶ Shao, Z., et al. (2024). *DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models* (GRPO). [arXiv:2402.03300](#).
- ▶ Nimmatur, D., et al. (2025). *Predictive Scaling Laws for Efficient GRPO Training of Large Reasoning Models*. [arXiv:2507.18014](#).
- ▶ OpenAI (2024). *Learning to Reason with LLMs*. [openai.com](#).

- ▶ Choi, Y. (2026). *CS224N Lecture 12: Reasoning Models*. [Stanford CS224N](#).
- ▶ CMU L3 Lab (2025). *Advanced NLP: Post-training & Meta-Generation*. [cmu-l3.github.io/anlp-spring2025](#).
- ▶ Lambert, N. *RLVR, scaling laws, reward saturation*. [natolambert.com/slides](#).
- ▶ Unsloth (2025). *Reinforcement Learning Guide (VRAM / QLoRA + RL)*. [unsloth.ai](#).
- ▶ Liu, Z., et al. (2025). *Understanding R1-Zero-Like Training: A Critical Perspective (Dr. GRPO, length bias)*. [arXiv:2503.20783](#).
- ▶ Wei, J., et al. (2022). *Chain-of-Thought Prompting Elicits Reasoning in LLMs*. [arXiv:2201.11903](#).
- ▶ Wang, X., et al. (2022). *Self-Consistency Improves Chain of Thought Reasoning*. [arXiv:2203.11171](#).